SMART CONTRACTS:
GAS

Course Leads:
Sara Magaziotis-Ginori
Olivia Li
Isaac Oh

Aman Shah
Elson Liu

Fujia Wang

BLOCKCHAIN
AT BERKELEY

0 Announcements

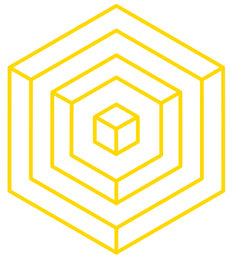
Announcements

- HW #4 (Etherjs) due today at 11:59PM
    - For extensions, email dev-decal@blockchain.berkeley.edu !
- HW #5 (Gas Optimization) due next Wednesday at 11:59PM

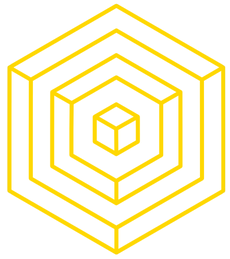
LECTURE OVERVIEW

1. Recap of Upgradeability
2. Overview of Gas in the EVM
3. Approaches to Reducing Gas
4. Solidity & Security
5. Solidity Resources

Special thanks to Doug Colkitt, whose guest lecture in Fall 2022 was adapted into these slides.



Recap
Upgradeability

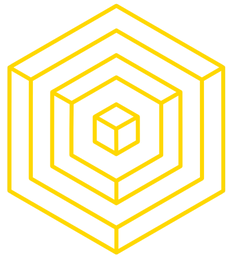


Example

Proxy Contract Pattern

```
// Proxy contract
contract Greetings {
  // Storage variable to store the address of the implementation contract.
  address private implementationAddress;

  // Constructor to initialize the implementation contract address.
  constructor(address implementation) {
    implementationAddress = implementation;
  }

  // Method to update the implementation contract address.
  function updateImplementation(address newImplementation) public {
    implementationAddress = newImplementation;
  }

  // Method to invoke the greetings function on the implementation contract.
  function greetings(string memory name) public returns (string memory) {
    // Forward the call to the implementation contract.
    return GreetingImplementation(implementationAddress).greetings(name);
  }
}
```

AUTHOR: DANIEL GUSHCHYAN
ADAPTED: DOUG COLKITT



Your Two Contracts

Example

```
// Implementation contract 1
contract GreetingImplementation1 {
  // Greetings function with customized message.
  function greetings(string memory name) public pure returns (string memory) {
    return string(abi.encodePacked("Hello, ", name, "! How are you today?"));
  }
}

// Implementation contract 2
contract GreetingImplementation2 {
  // Greetings function with another customized message.
  function greetings(string memory name) public pure returns (string memory) {
    return string(abi.encodePacked("Hello, ", name, "! How can I help you today?"))
  }
}
```

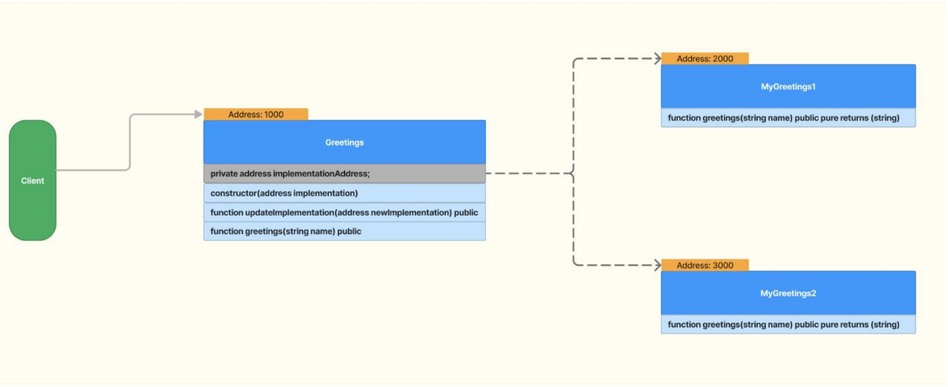
AUTHOR: DANIEL GUSHCHYAN
ADAPTED: DOUG COLKITT

Explain:)

```
// Proxy contract
contract Greetings {
  // Storage variable to store the address of the implementation contract.
  address private implementationAddress;

  // Constructor to initialize the implementation contract address.
  constructor(address implementation) {
    implementationAddress = implementation;
  }

  // Method to update the implementation contract address.
  function updateImplementation(address newImplementation) public {
    implementationAddress = newImplementation;
  }

  // Method to invoke the greetings function on the implementation contract.
  function greetings(string memory name) public returns (string memory) {
    // Forward the call to the implementation contract.
    return GreetingImplementation(implementationAddress).greetings(name);
  }
}
```

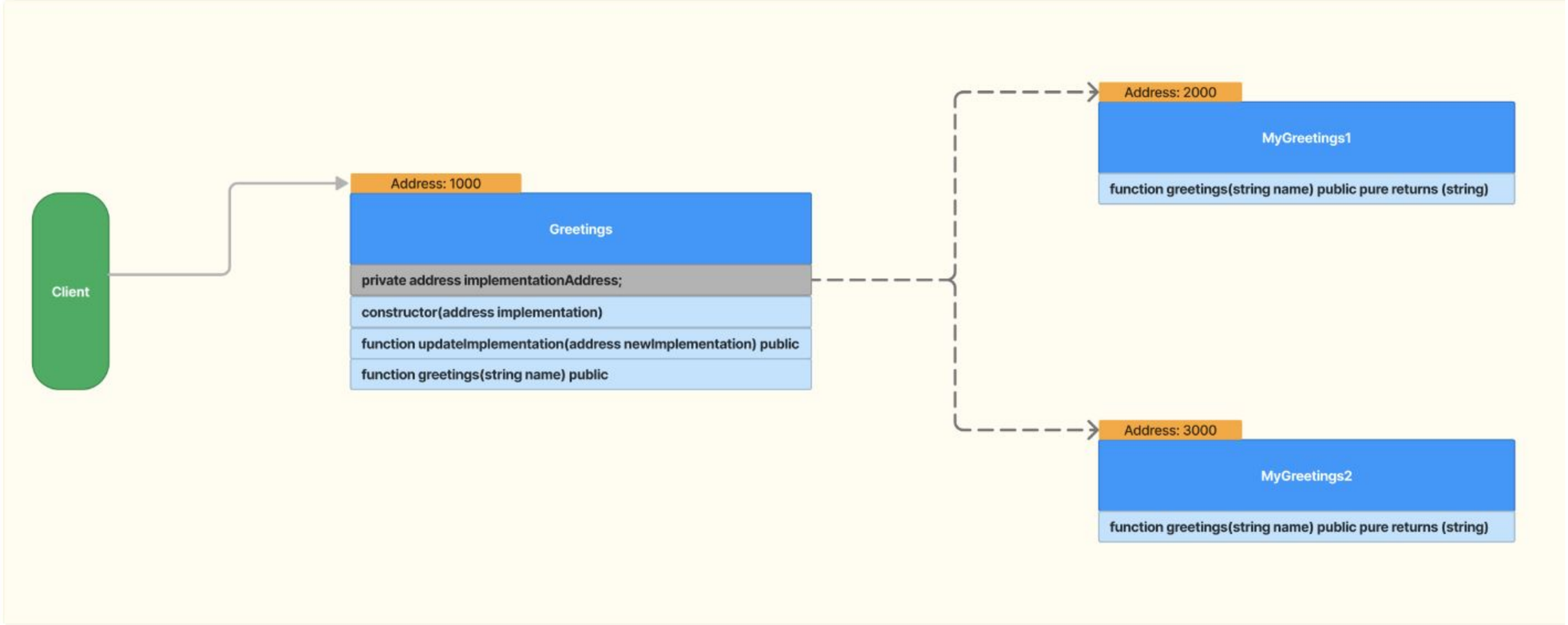
```
// Implementation contract 1
contract GreetingImplementation1 {
  // Greetings function with customized message.
  function greetings(string memory name) public pure returns (string memory) {
    return string(abi.encodePacked("Hello, ", name, "! How are you today?"));
  }
}
```

```
// Implementation contra
contract GreetingImpleme
  // Greetings function
  function greetings(str
    return string(abi.e
  }
}
```



AUTHOR: DANIEL GUSHCHYAN
ADAPTED: DOUG COLKITT

Answer



AUTHOR: DANIEL GUSHCHYAN
ADAPTED: DOUG COLKITT

How to Use Them

- **We aren't gonna go into too much detail, if you're interested, read more here**
- Here are the commands to generate an openZeppelin upgradeable contract

Bash

Copy Caption

```
npm install @openzeppelin/contracts-upgradeable
```

Solidity

Copy Caption

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "@openzeppelin/contracts-upgradeable/token/ERC20/ERC20Upgradeable.sol";
import "@openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol";

contract MyToken is Initializable, ERC20Upgradeable {
    function initialize() public initializer {
        __ERC20_init("MyToken", "MTK");
        _mint(msg.sender, 10000 * 10 ** decimals());
    }
}
```

AUTHOR: DANIEL GUSHCHYAN
ADAPTED: DOUG COLKITT

1 Overview of Gas in the EVM

GAS OVERVIEW

RECALL..

What is Gas?

- Gas represents units of **work** done by nodes in the network when processing transactions
  - State growth
  - Memory
  - Bandwidth
    Compute

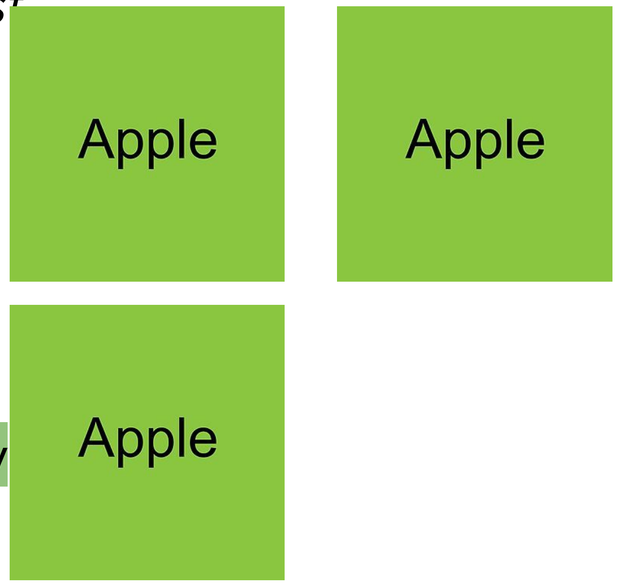
- Ethereum protocol sets a **gas limit** to prevent abuse or network overload
- Gas attempts to approximate many dimensions of work with one measurement

AUTHOR: DANIEL GUSHCHYAN
ADAPTED: DOUG COLKITT

GAS CALCULATIONS

HOW IS GAS CHARGED

The amount of ether you pay for gas is defined by two numbers: gas *cost* and gas *price*.



Gas Cost

- Every operation in every transaction has a gas cost
- Each transaction has a gas usage and a gas price
- Gas usage comes from the code in the smart contract
- Example: it takes 3 apples to make an apple pie. This is determined by the apple pie recipe.

Gas Price (gwei)

- Gas price is set by the user
  - Higher gas price - faster
  - Lower gas price - cheaper
  - Example: one apple may cost $10 or $1 depending on how much demand there is for apples. If everyone wants an apple, you can pay more than other people to get the apple like $20.

AUTHOR: ALBERT SU
UPDATED: OLIVIA LI

WHY CARE ABOUT GAS?

JUST DEPLOY THE CONTRACT, BRO

It is your users that pay for the gas!

The most used smart contracts cost users tens of millions of dollars per year.

Top 50 Gas Guzzlers (Contracts / Accounts that consume a lot of Gas)

Last updated at Block 16843982

Rank	Address	Fees Last 3hrs	% Used 3hrs	Fees Last 24hrs	% Used 24hrs	Analytics
1	Uniswap: Universal Router	$52,595.40 (31.48 Eth)	10.26%	$475,207.80 (284.42 Eth)	10.73%	
2	0x322169...EE31A6AF	$63,831.59 (38.20 Eth)	6.76%	$63,831.59 (38.20 Eth)	0.85%	
3	Blur.io: Marketplace	$29,637.58 (17.74 Eth)	5.99%	$234,537.48 (140.37 Eth)	5.40%	
4	Seaport 1.4	$24,700.98 (14.78 Eth)	5.04%	$177,976.75 (106.52 Eth)	4.14%	
5	Tether: USDT Stablecoin	$22,464.16 (13.44 Eth)	3.92%	$245,737.22 (147.08 Eth)	5.03%	

The more efficient the contract, the less your users pay to use it.

Also, if you want to MEV (maximal extractable value).

AUTHOR: DANIEL GUSHCHYAN
ADAPTED: DOUG COLKITT

2 Approaches to Reducing Gas

GAS BY NUMBER

Memorize this stuff!

Assuming ETH price of $4480 and gas price of .39 gwei (if the network is more congested, more gwei)

Operation	Typical gas units	Cost (USD)	Notes
Arithmetic op (+, –, ×, ÷)	~3	**$0.000005**	basically free
Function call argument	~700	**$0.0012**	less than a cent
Contract call	~37 k	**$0.065**	a few cents
Reading from storage (SLOAD)	~21 k	**$0.037**	mostly read cost
Updating storage (SSTORE)	~100 k	**$0.17**	writing still pricy
Transfer	~21 k	**$0.037**	simple ETH transfer
Uniswap swap	~120 k–200 k	**$0.21–$0.35**	depends on pool complexity
Max contract deployment	~2 M	**$3.50**	large contracts still cost dollars

AUTHOR: ALBERT SU
UPDATED: OLIVIA LI

SIMPLE OPTIMIZATION

Memory/Storage Optimzations

- The EVM wants to reduce state bloat(every full node needs to hold all the data)
  - Thus, **writing data is the most expensive operation** you can do
-
- Deleting storage(aka zeroing stuff out) pays a gas refund
  - Caveat: Only half of the pre-refunded gas can actually be refunded bc of gas tokens
- Don’t use anything bigger than 32 bytes in memory(arrays, strings, structs, bytes)
  - Variables < 32 bytes are stored on the stack(basically free)

$$C_{\text{mem}}(a) \equiv G_{\text{memory}} \cdot a + \left\lfloor \frac{a^2}{512} \right\rfloor$$

- Or, use calldata instead of memory in function parameters
  - Calldata is cheaper, but static and can’t be modified in the transaction

- Use external/internal instead of public modifiers
  - Public modifiers copy function parameters into memory

AUTHOR: ALBERT SU
ADAPTED: DOUG COLKITT

SIMPLE OPTIMIZATION

If you don’t want to mess with assembly, do this

- Use gas optimized libraries: Solmate is what you are looking for
- Try to short circuit code such that you don’t run through the entire function body
- Move computation offchain
- Make commonly used functions cheap and rarely used functions expensive
- Use the solidity optimizer
  - If you maximize runs, it increases the deployment cost but decreases the cost per call for functions
  - If you set runs=1, it minimizes deployment costs but increases the cost per call
- Struct packing: Group variables into 256 bit chunks
- Map packing: One map tracking a struct of 10 variables is cheaper than 10 maps tracking the 1 variable each
- Remove all looping/array functions

AUTHOR: ALBERT SU
ADAPTED: DOUG COLKITT

SIMPLE OPTIMIZATION

Misc Optimzations

- Zeroes are cheaper than nonzero values
  - 0xFFFF is more expensive to pass in than 0x0
  - 0x1000 is cheaper than 0x1111
- Payable functions are cheaper than non-payable functions
  - Payable removes an internal msg.value check
- Mark arithmetic as unchecked when you know FOR SURE it is safe
  - Solidity automatically adds overflow checks to every arithmetic operation
  - for (uint256 i = 0; i < length;) {
  - // ...
  - unchecked { i += 1; }
- Mark functions as internal if they aren't meant to be publicly called
  - Lets the solidity optimizer inline the code for gas savings
- < is cheaper than <=
- == is cheaper than !=

AUTHOR: ALBERT SU
ADAPTED: DOUG COLKITT

3 Detailed Optimizations

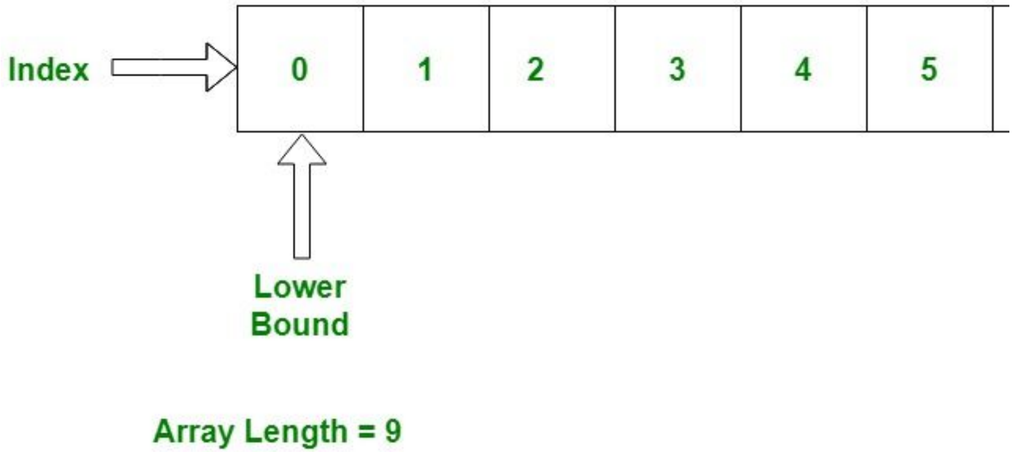
Arithmeticcccc

- Arithmetic Operations:
  - Types: Multiplication, Division
  - Gas Cost: 5 units
  - Consideration: Reverts on overflow
- Bitwise Operations:
  - Types: AND, OR, XOR
  - Gas Cost: 3 units
  - Advantage: More gas-efficient compared to arithmetic operations
- Shift Operators:
  - Types: << (left shift), >> (right shift)
  - Characteristic: Do not revert on overflow
  - Classification: Not considered arithmetic operations
- Best Practices:
  - Assess the necessity of arithmetic operations and replace with bitwise operations where applicable.
  - Prioritize bitwise operations to conserve gas.
  - Utilize shift operators for scenarios where overflow management is crucial.

AUTHOR: Naman Kapasi
ADAPTED: DOUG COLKITT

ARRAYYYSS

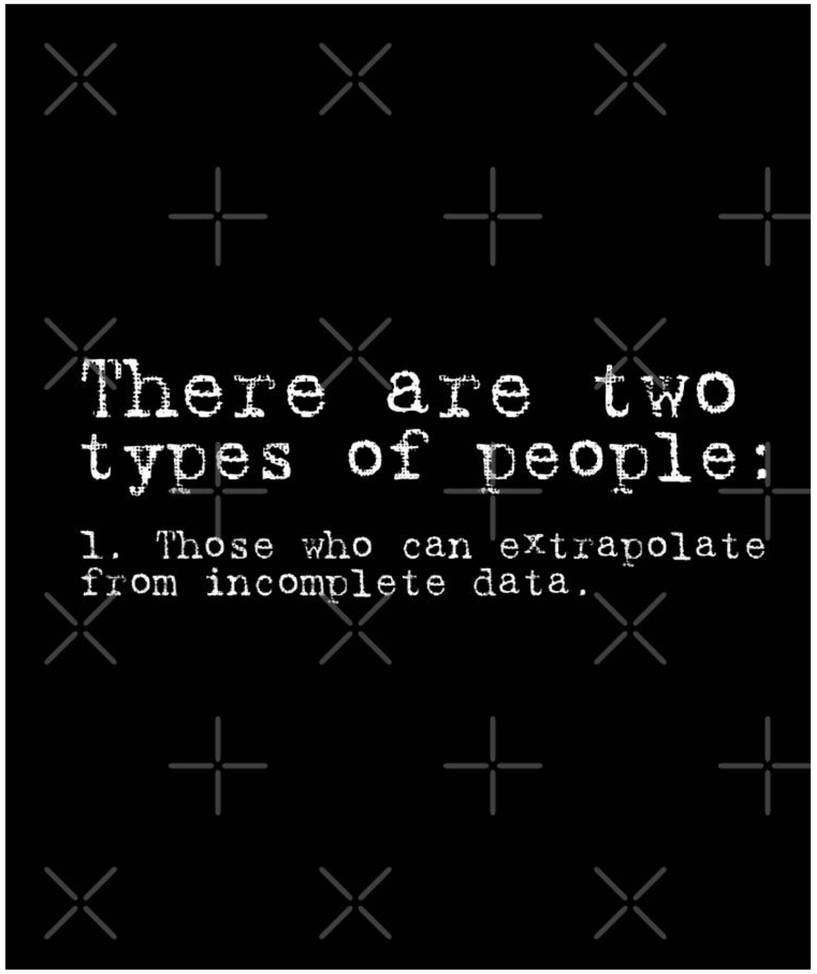
- Problem:
  - Reading the .length property of an array within a loop causes multiple SLOAD (storage read) operations, increasing gas cost.
  - Unnecessary incrementation of the loop variable in each iteration.
- Solution:
  - Store the length of the array in a local variable before the loop starts to reduce SLOAD operations to one.
  - Increment the loop variable only once per iteration to enhance loop efficiency.
- Benefits:
  - Significant reduction in gas cost, especially beneficial when the array is large or the function is invoked multiple times.
  - Improved performance and efficiency of the function.



AUTHOR: Naman Kapasi
ADAPTED: DOUG COLKITT

Data Types

- It is better to use uint256 and bytes32 than, for example, uint8. While it seems like uint8 will consume less gas than uint256, this is not true since the Ethereum Virtual Machine (EVM) will still occupy 256 bits, fill 8 bits with the uint variable, and fill the extra bits with zeros.
- External calls are expensive, therefore, fewer external calls == fewer gas cost.
- Also, consider appropriate function visibility. Internal functions are cheaper to call than public functions. Therefore, there is no need to mark a function as public if it is only meant to be called internally.
- There's no garbage collection in Solidity. Thus, we will have to throw away or delete the unused data on our own.



AUTHOR: Naman Kapasi
ADAPTED: DOUG COLKITT

More Fun Stuff

++i costs less gas compared to i++ or i += 1

Pre-increments and pre-decrements are cheaper.

Increment:

- i += 1 is the most expensive form
- i++ costs 6 gas less than i += 1
- ++i costs 5 gas less than i++ (11 gas less than i += 1)

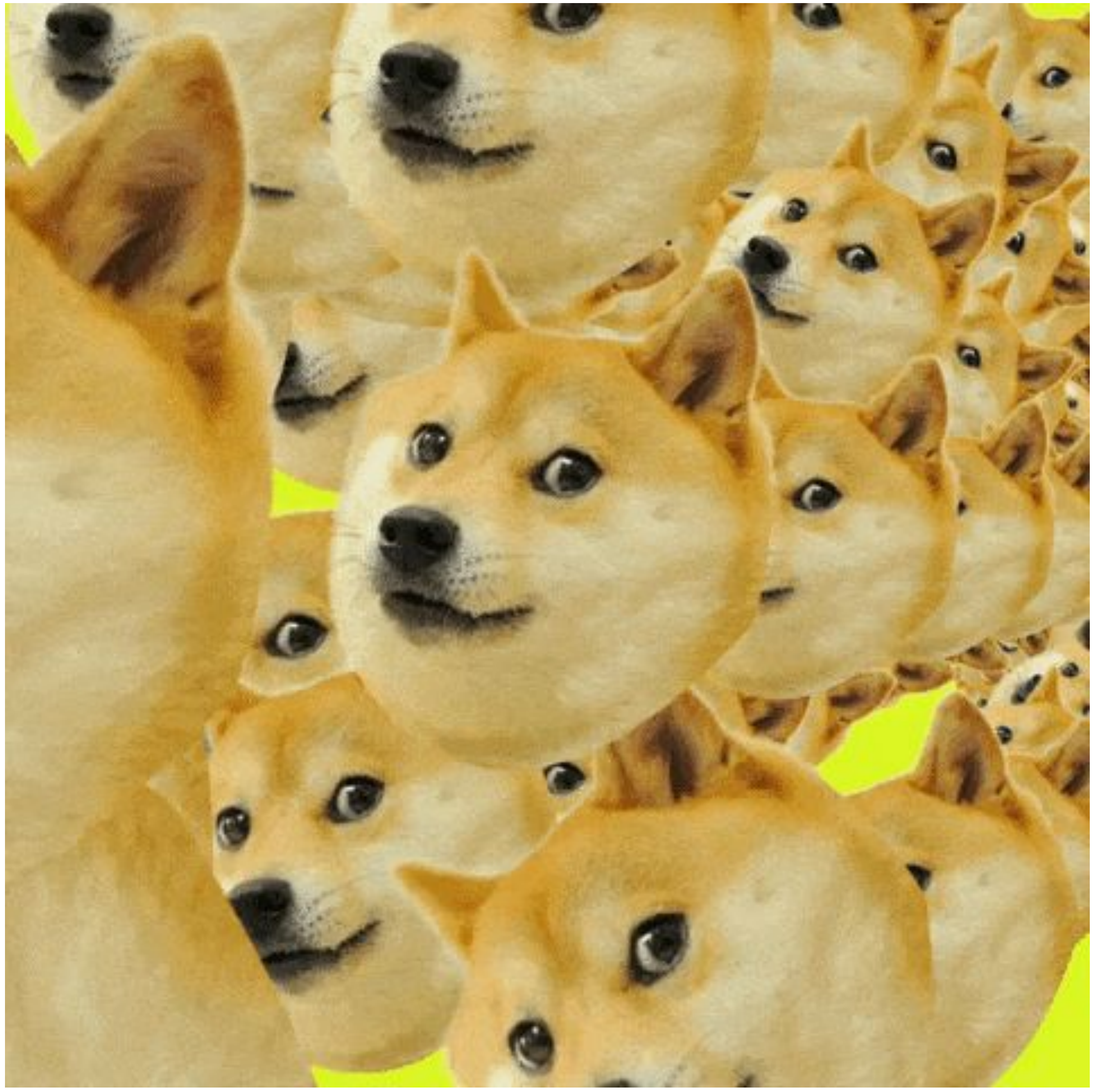
Decrement:

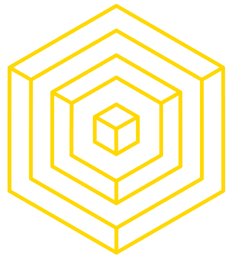
- i -= 1 is the most expensive form
- i-- costs 11 gas less than i -= 1
- --i costs 5 gas less than i-- (16 gas less than i -= 1)

Note that post-increments (or post-decrements) return the old value before incrementing or decrementing, hence the name post-increment:

AUTHOR: Naman Kapasi
ADAPTED: DOUG COLKITT

CLICK HERE

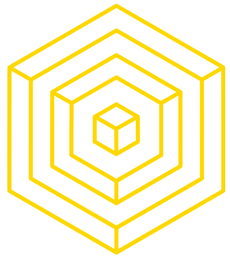




3 Solidity Trivia

Random facts to flex with

BLOCKCHAIN AT BERKELEY



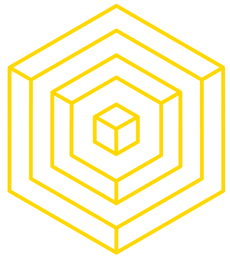
CONTRACTING LIKE A PRO

I just wrote a contract, what's next

Assuming ETH price of 1500 and gas price of 20 gwei:

- Use natspec comments
- Look into delegatecall/staticcall/call
- Look into ecrecover
- Look into try/catch

AUTHOR: ALBERT SU
ADAPTED: DOUG COLKITT



4 Solidity & Security

WHY CARE?

You'll sleep better at night if your contracts are secure

Lots of people teach what to do, since idk what to do, let's go through what not to do

- Key exploit: Ronin bridge - 624m
- Cryptography exploit: BNB bridge - 586m
- Simple code bug: Wormhole - 326m
- Logic errors: Euler Finance - 197m
- Governance Attack: Beanstalk - 181m
- Address Pitfalls: Wintermute - 162m
- Frontend: BadgerDAO - 120m
- Price Manipulation: Mango Markets - 115m
- Re-entrancy: Fei/Rari - 80m

Note: Flash loans are used in many hacks, I didn't add it as a separate category

AUTHOR: ALBERT SU
ADAPTED: DOUG COLKITT

5 Solidity Resources

Solidity Developer Communities

Where do they hang out on the internet

If you can’t google them, send me an email and I’ll send you the invite links

- Discords: Secureum, Spearbit, Tenderly, Hardhat, Flashbots
- Telegram: LobsterDAO, ETH Security, Foundry Support

AUTHOR: ALBERT SU
ADAPTED: DOUG COLKITT

Solidity Tooling

- Hardhat
  - The OG solidity tool - supports testing, deployments, libraries, basically everything
  - Javascript/Typescript Based
- Foundry
  - Hardhat but better, Solidity based
- Security Tooling
  - https://github.com/nascentxyz/pyrometer
  - https://github.com/trailofbits/eth-security-toolbox

AUTHOR: ALBERT SU
ADAPTED: DOUG COLKITT

6 This Week in Crypto

Article

- https://www.coindesk.com/markets/2024/02/21/nvidias-hotly-anticipated-earnings-may-trigger-bitcoin-and-crypto-correction-analyst-says/?_gl=1*703bqi*_up*MQ..*_ga*MTgzNTM2NDY0NS4xNzA4NTY3MDUw*_ga_VM3STRYVN8*MTcwODU2NzA0OS4xLjAuMTcwODU2NzA0OS4wLjAuMA..
- Nvidia’s q4 earnings report may affect the price of the cryptocurrency market

VITAMIN/ATTENDANCE